ODONTOCLINIC

Documentación Técnica del Sistema

Proyecto de Seminario

Ingeniería en Sistemas

Universidad Mariano Gálvez de Guatemala

Estudiante: [Completar información]

Carné: [Completar información]

Docente: [Completar información]

Sede: [Completar información]

Fecha: [Completar información]

Control del documento

Campo	Valor
Nombre del documento	Documentación Técnica de OdontoClinic
Sistema	OdontoClinic
Versión del documento	1.0
Fecha de elaboración	5 de octubre de 2026
Estado	Documentación técnica del sistema
Versiones declaradas del sistema	Frontend 0.0.0; backend 0.0.1

Este documento presenta la estructura, el funcionamiento y la configuración técnica verificable de OdontoClinic. Las consideraciones y recomendaciones se distinguen de las capacidades implementadas para conservar la precisión técnica.

Tabla de contenido

Control del documento 2
Tabla de contenido 3
1 Introducción 6
2 Objetivo de la documentación 6
3 Alcance del sistema 6
3.1 Funcionalidades implementadas 6
3.2 Consideraciones sobre la implementación actual 6
3.3 Componentes existentes sin integración funcional demostrada 7
4 Descripción general del sistema 7
5 Arquitectura del sistema 8
5.1 Responsabilidades por capa 8
5.2 Consideraciones de la arquitectura actual 8
6 Stack tecnológico 9
7 Estructura del proyecto 9
8 Arquitectura del Frontend 10
8.1 Rutas 10
8.2 Servicios y comunicación 10
8.3 Estado y almacenamiento 10
8.4 Manejo de errores frontend 10
9 Arquitectura del Backend 11
9.1 Configuración técnica actual del backend 11
10 Módulos funcionales 11
10.1 Autenticación 11
10.2 Dashboard 12
10.3 Agenda y citas 12
10.4 Pacientes 12
10.5 Historia clínica 12
10.6 Reportes 13
10.7 Usuarios y perfil 13
11 Autenticación y autorización 13
11.1 Mecanismo de autenticación 14
11.2 Cobertura actual de autenticación en el backend 14
11.3 Expiración, refresh y logout 14
12 Roles y permisos 14
13 API REST 15

13.1 Raíz y autenticación 15
13.2 Usuarios 15
13.3 Pacientes 15
13.4 Citas 15
13.5 Historia clínica 16
13.6 Cuerpos principales 16
14 Base de datos 16
14.1 Enums 17
15 Diccionario de datos 17
15.1 Usuario 17
15.2 Paciente 17
15.3 Cita 18
15.4 HistoriaClinica 18
16 Modelo entidad relación 19
17 Flujos principales del sistema 19
17.1 Inicio de sesión 19
17.2 Alta de paciente 20
17.3 Programación de cita 20
17.4 Consulta de historia 20
17.5 Generación de reporte 21
18 Seguridad 21
18.1 Controles implementados 21
18.2 Consideraciones de seguridad 21
19 Configuración del sistema 22
20 Variables de entorno 22
21 Instalación y ejecución local 22
21.1 Prerrequisitos verificables 22
21.2 Backend 23
21.3 Frontend 23
21.4 Build 23
22 Scripts disponibles 23
22.1 Frontend 23
22.2 Backend 23
23 Despliegue 24
24 Manejo de errores 24
25 Validaciones 24

26 Pruebas 25
27 Dependencias principales 25
28 Consideraciones de mantenimiento 25
29 Aspectos técnicos pendientes de fortalecimiento 26
30 Recomendaciones técnicas 26
31 Información pendiente de documentación 27
32 Conclusiones técnicas 27
Anexo A Trazabilidad de evidencia 27
Anexo B Resumen cuantitativo del proyecto 28

En Microsoft Word, seleccione la tabla y use Actualizar campo para recalcular títulos y páginas después de editar el documento.

1 Introducción

OdontoClinic es una aplicación web orientada a la gestión clínica odontológica. Permite administrar usuarios, pacientes y citas; consultar información histórica construida a partir de las citas; y generar reportes exportables. Su arquitectura combina un frontend Angular, una API desarrollada con NestJS y una capa de persistencia implementada con Prisma sobre PostgreSQL.

La presente documentación explica la organización y el funcionamiento del sistema a partir del código disponible. También registra consideraciones técnicas y oportunidades de mejora relacionadas con integración, autenticación, autorización, validación, pruebas y despliegue, sin atribuir al proyecto capacidades que no estén implementadas.

2 Objetivo de la documentación

El objetivo de esta documentación es facilitar la comprensión, instalación, mantenimiento y continuidad del desarrollo de OdontoClinic. Para ello describe la arquitectura, los módulos funcionales, el frontend, el backend, el modelo de datos, los contratos HTTP, la autenticación, los roles, la configuración, la ejecución local y las referencias de despliegue disponibles. Asimismo, proporciona una base técnica para soporte futuro y presenta oportunidades de fortalecimiento separadas de las funciones que el sistema ofrece actualmente.

3 Alcance del sistema

3.1 Funcionalidades implementadas

- Inicio de sesión con correo y contraseña, emisión de un JWT y almacenamiento de la respuesta en localStorage.
- Dashboard con métricas derivadas de pacientes y citas de una fecha seleccionada.
- Agenda con vistas diaria, semanal y mensual; alta, edición, cambio de estado y eliminación de citas.
- Alta, consulta, edición, filtrado, paginación y eliminación condicionada de pacientes.
- Consulta de perfil de paciente e historial construido con citas asociadas.
- Reportes de pacientes, citas e historial clínico agregados en el navegador y exportables a CSV, XLSX y PDF.
- Gestión de usuarios desde la interfaz, con roles ADMIN, DOCTOR y RECEPCIONISTA.
- API CRUD para usuarios, pacientes, citas e historias clínicas, además de registro, login, perfil y actualización autenticada.

3.2 Consideraciones sobre la implementación actual

- El frontend denomina historia clínica a una vista de citas y notas; no consume el CRUD backend de HistoriaClinica ni muestra diagnóstico, tratamiento u observaciones persistidos en esa entidad.
- El servicio de reportes contiene rutas HTTP hipotéticas dentro de una rama desactivada. El backend no implementa un controlador de reportes; la ruta activa agrega datos en el cliente.
- Los parámetros date, search y pacienteId se envían desde el frontend, pero los controladores backend no los reciben ni procesan. La aplicación vuelve a filtrar en el cliente.
- Existe un RolesGuard y un decorador Roles, pero ningún controlador los aplica.
- El campo refreshToken existe en el esquema, pero el frontend lo utiliza como identificador del doctor asignado a una recepcionista. No existe una operación de renovación de JWT.

3.3 Componentes existentes sin integración funcional demostrada

- Las clases de entidad TypeScript de Usuario, Paciente, Cita e HistoriaClinica están vacías.
- frontend/src/app/pacientes/temp.service.ts es una copia incompleta que no aparece importada por la aplicación compilada.
- backend/dev.db tiene tamaño cero; el esquema activo configura PostgreSQL y no referencia ese archivo.
- GET / y sus pruebas conservan la respuesta de plantilla Hello World!.

4 Descripción general del sistema

El flujo habitual comienza cuando el usuario inicia sesión y accede al dashboard. Desde la navegación principal puede consultar y administrar pacientes, programar o actualizar citas, revisar el historial disponible, generar reportes y, de acuerdo con las opciones visibles para su rol, gestionar usuarios y su perfil.

Este flujo se implementa mediante una aplicación Angular basada en componentes standalone. Las rutas funcionales utilizan authGuard, que comprueba la existencia de access_token en localStorage. Los servicios de dominio consumen la API con HttpClient mediante una URL configurada directamente en el frontend, y un interceptor agrega el encabezado Authorization Bearer cuando encuentra un token.

NestJS recibe las solicitudes en controladores por recurso y delega en servicios que ejecutan operaciones Prisma. Prisma obtiene la conexión PostgreSQL desde DATABASE_URL. La API no establece un prefijo global, por lo que los paths documentados parten de la raíz del servidor.

5 Arquitectura del sistema

OdontoClinic utiliza una arquitectura cliente servidor. Angular proporciona la capa de presentación y comunicación HTTP; NestJS organiza la API, la autenticación y los servicios de dominio; Prisma traduce las operaciones de los servicios a consultas sobre los modelos; y PostgreSQL conserva los datos y sus relaciones.

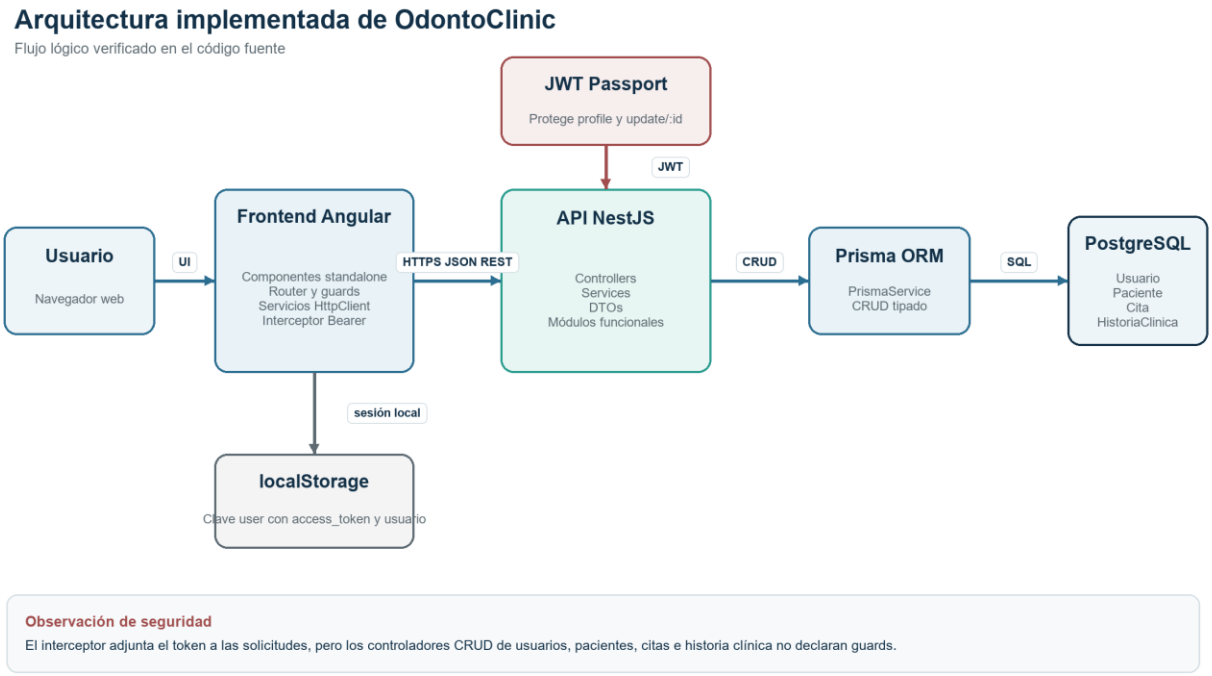


Figura 1. Arquitectura lógica reconstruida desde el código fuente.

5.1 Responsabilidades por capa

Capa	Responsabilidad verificada	Evidencia principal
Presentación	Vistas, formularios, filtros, navegación y exportación de reportes.	frontend/src/app
Cliente HTTP	Mapeo de modelos, transformación de fechas, filtros locales y consumo REST.	Servicios Angular por módulo
API	Rutas HTTP, autenticación, servicios CRUD y respuestas de NestJS.	backend/src
Acceso a datos	PrismaClient global, conexión y operaciones sobre modelos.	backend/src/prisma
Persistencia	Tablas, enums, restricciones y relaciones PostgreSQL.	backend/prisma/schema.prisma y migración

5.2 Consideraciones de la arquitectura actual

La implementación distribuye algunas responsabilidades entre cliente y servidor. El frontend aplica filtros de alcance por usuario, resuelve nombres de pacientes con solicitudes adicionales y consolida los reportes en memoria. También mantiene la URL de la API directamente en los servicios. La vista de historia clínica utiliza las citas asociadas, mientras que el backend dispone de una entidad HistoriaClinica independiente. Estas características deben considerarse al extender la arquitectura o trasladar reglas al servidor.

6 Stack tecnológico

Tecnología	Versión comprobada	Uso	Evidencia
Angular	19.2.25	SPA y componentes standalone	frontend/package-lock.json
Angular CLI	19.2.27	Build, serve y pruebas	frontend/package-lock.json
TypeScript frontend	5.7.3	Código del cliente	frontend/package-lock.json
Bootstrap	5.3.8	Estilos y modales	frontend/package-lock.json y index.html
RxJS	7.8.2	Flujos HTTP y estado asíncrono	package-lock de ambos componentes
ExcelJS	4.4.0	Exportación XLSX	frontend/package-lock.json
jsPDF	3.0.4	Exportación PDF	frontend/package-lock.json
jsPDF AutoTable	5.0.8	Tablas en PDF	frontend/package-lock.json
NestJS	11.1.6	API HTTP modular	backend/package-lock.json
Prisma Client	6.17.1	ORM y acceso a datos	backend/package-lock.json
PostgreSQL	No fijada	Motor definido por el datasource	backend/prisma/schema.prisma
bcrypt	6.0.0	Hash y comparación de contraseñas	backend/package-lock.json
Passport JWT	4.0.1	Extracción y validación Bearer	backend/package-lock.json
Jest	30.1.3	Pruebas backend	backend/package-lock.json
Karma	6.4.4	Pruebas frontend	frontend/package-lock.json

7 Estructura del proyecto

El repositorio mantiene frontend y backend como proyectos npm independientes. El package.json raíz solo declara @nestjs/config y no coordina scripts de ambos componentes.

```
odonto_clinic/
├─ frontend/
│  ├─ src/app/                componentes, rutas, guards y servicios
│  ├─ src/assets/             recursos SVG
│  ├─ angular.json            build, test y deploy gh-pages
│  └─ package.json            scripts y dependencias del cliente
├─ backend/
│  ├─ src/auth/               login, JWT, DTOs y guards
│  ├─ src/{usuarios,pacientes,citas,historia-clinica}/
│  ├─ src/prisma/             módulo y servicio Prisma
│  ├─ prisma/                 schema y migración inicial
│  ├─ test/                   prueba e2e de la ruta raíz
│  └─ package.json            scripts y dependencias de la API
└─ package.json               dependencia raíz sin scripts
```

8 Arquitectura del Frontend

El frontend usa bootstrapApplication y componentes standalone. app.config.ts registra el router, HttpClient con AuthInterceptor y coalescencia de eventos. Las rutas login, dashboard, agenda, pacientes, historia clínica, reportes, configuración y perfil se declaran en app.routes.ts.

8.1 Rutas

Ruta	Componente	Carga	Protección
/login	LoginComponent	Lazy	loginGuard evita mostrarla si existe token
/dashboard	DashboardComponent	Directa	authGuard
/agenda	AgendaComponent	Directa	authGuard
/pacientes	PacientesComponent	Directa	authGuard
/pacientes/:id	PacientePerfilComponent	Lazy	authGuard
/historia-clinica	HistoriaClinicaComponent	Directa	authGuard
/reportes	ReportesComponent	Directa	authGuard
/configuracion	ConfiguracionComponent	Directa	authGuard; no guard de rol
/perfil	PerfilComponent	Lazy	authGuard
**	Redirección	-	Envía a /dashboard

8.2 Servicios y comunicación

Servicio	Responsabilidad	API activa
AuthService	Login, logout, token y usuario actual	https://odonto-clinic.onrender.com
AgendaService	CRUD de citas, fechas, filtros y nombres de pacientes	Misma URL
PacientesService	CRUD, mapeo de fechas y alcance por usuario	Misma URL
HistoriaClinicaService	Combina paciente con citas filtradas	Misma URL
UsuariosService	Gestión de usuarios	Misma URL
ReportesService	Agrega datos en el cliente	No usa /reportes mientras useClientAggregation sea true

8.3 Estado y almacenamiento

La aplicación usa signals y computed para estado de interfaz. La autenticación se guarda como JSON bajo la clave user de localStorage. El interceptor lee access_token de esa estructura. Los almacenes mock dentalpro_pacientes_v1 y dentalpro_usuarios_v1 permanecen en servicios, pero useMock está configurado en false.

8.4 Manejo de errores frontend

Los componentes gestionan errores mediante callbacks de subscribe y mensajes de interfaz. Algunos servicios utilizan catchError para retornar arreglos vacíos o valores undefined. HistoriaClinicaService aplica

un timeout de 10 segundos y, ante un error, devuelve una estructura de paciente y citas vacías; este comportamiento permite mantener la vista operativa, aunque reduce la información disponible para diagnóstico.

9 Arquitectura del Backend

AppModule importa UsuariosModule, PacientesModule, CitasModule, HistoriaClinicaModule y AuthModule. PrismaModule es global y expone PrismaService. Cada módulo CRUD organiza el flujo desde Controller hacia Service y PrismaService. La implementación trabaja directamente con PrismaService, sin una capa repository adicional.

Módulo	Controlador	Servicio	Persistencia
Auth	AuthController	AuthService	Usuario mediante Prisma
Usuarios	UsuariosController	UsuariosService	Usuario
Pacientes	PacientesController	PacientesService	Paciente
Citas	CitasController	CitasService	Cita
Historia clínica	HistoriaClinicaController	HistoriaClinicaService	HistoriaClinica
Prisma	-	PrismaService	Conexión y desconexión del cliente

9.1 Configuración técnica actual del backend

- NestFactory crea AppModule y escucha PORT o 3000.
- CORS permite http://localhost:4200, https://yorchriv.github.io y https://odonto-clinic.onrender.com, con credenciales y métodos CRUD.
- La configuración actual no registra ValidationPipe global, prefijo global, Swagger, Helmet, rate limiting ni un filtro global de excepciones.

10 Módulos funcionales

10.1 Autenticación

Elemento	Descripción
Finalidad	Validar credenciales, emitir JWT y proporcionar datos del usuario autenticado.
Componentes y servicios	LoginComponent, AuthService de Angular, AuthController, AuthService de NestJS, JwtStrategy y JwtAuthGuard.
Endpoints	POST /auth/register, POST /auth/login, GET /auth/profile y PUT /auth/update/:id.
Entidad asociada	Usuario.
Flujo funcional	El formulario envía credenciales; el backend valida estado y contraseña con bcrypt, firma el token y devuelve la identidad al frontend.

10.2 Dashboard

Elemento	Descripción
Finalidad	Presentar métricas operativas y accesos rápidos después del inicio de sesión.
Componentes y servicios	DashboardComponent, AgendaService y PacientesService.
Endpoints	Consume las consultas de citas y pacientes; no dispone de un endpoint propio.
Entidades asociadas	Paciente y Cita.
Flujo funcional	El componente combina respuestas de los servicios y calcula conteos según la fecha seleccionada.

10.3 Agenda y citas

Elemento	Descripción
Finalidad	Programar, consultar, actualizar, finalizar y cancelar citas.
Componentes y servicios	AgendaComponent, AgendaService, CitasController y CitasService.
Endpoints	CRUD de /citas.
Entidad asociada	Cita, relacionada con Paciente y Usuario.
Flujo funcional	La interfaz prepara fecha, hora, paciente y usuario; envía la solicitud y aplica filtros por usuario y periodo en el cliente.

10.4 Pacientes

Elemento	Descripción
Finalidad	Administrar datos personales, estado y alergias de los pacientes.
Componentes y servicios	PacientesComponent, PacientePerfilComponent, PacientesService, PacientesController y PacientesService de NestJS.
Endpoints	CRUD de /pacientes.
Entidad asociada	Paciente, con relaciones hacia Usuario, Cita e HistoriaClinica.
Flujo funcional	La interfaz valida y transforma los datos, asigna creadoPorId y consume la API; el backend persiste mediante Prisma.

10.5 Historia clínica

Elemento	Descripción
Finalidad	Consultar antecedentes asociados al paciente y disponer de un modelo persistente para información clínica.
Componentes y servicios	HistoriaClinicaComponent, HistoriaClinicaService de Angular, HistoriaClinicaController y servicio backend.
Endpoints	CRUD de /historia-clinica; la vista actual consulta pacientes y citas en lugar de consumir ese CRUD.
Entidad asociada	HistoriaClinica, relacionada con Paciente y Cita.

Elemento	Descripción
Flujo funcional	La interfaz selecciona un paciente y construye el historial con citas y notas; diagnóstico, tratamiento y observaciones de la entidad backend no se muestran actualmente.

10.6 Reportes

Elemento	Descripción
Finalidad	Consolidar información de pacientes, citas e historial y exportarla.
Componentes y servicios	ReportesComponent y ReportesService.
Endpoints	No existe controlador backend de reportes; el flujo activo reutiliza servicios de pacientes, citas e historia.
Entidades asociadas	Paciente y Cita; datos de historia construidos por el frontend.
Flujo funcional	ReportesService agrega datos en el navegador y permite exportar CSV, XLSX y PDF.

10.7 Usuarios y perfil

Elemento	Descripción
Finalidad	Gestionar cuentas, roles, datos de perfil y asignación operativa de recepcionistas.
Componentes y servicios	ConfiguracionComponent, PerfilComponent, UsuariosService de Angular, UsuariosController y UsuariosService de NestJS.
Endpoints	CRUD de /usuarios y PUT /auth/update/:id.
Entidad asociada	Usuario.
Flujo funcional	La configuración administra cuentas; el perfil actualiza datos y contraseña. El campo refreshToken se utiliza en la interfaz como identificador del doctor asignado.

11 Autenticación y autorización



Figura 2. Secuencia exitosa de inicio de sesión.

11.1 Mecanismo de autenticación

1. LoginComponent valida email y contraseña con Angular Validators.
2. AuthService envía POST /auth/login.
3. AuthService del backend busca Usuario por email, verifica activo y ejecuta bcrypt.compare.
4. JwtService firma un payload con email, sub, rol, nombre y apellido; el login establece expiresIn en 24h.
5. El frontend guarda la respuesta completa en localStorage y navega a /dashboard.
6. AuthInterceptor adjunta el token como Bearer a solicitudes posteriores.

11.2 Cobertura actual de autenticación en el backend

JwtAuthGuard está aplicado únicamente a GET /auth/profile y PUT /auth/update/:id. La segunda ruta permite actualizar al propio usuario o a cualquier usuario cuando req.user.rol es ADMIN. Los controladores de usuarios, pacientes, citas e historia clínica no declaran UseGuards.

11.3 Expiración, refresh y logout

El token emitido por login expira en 24 horas. AuthModule también declara signOptions de 7 días, pero login reemplaza ese valor al firmar. No existe endpoint de refresh. El campo refreshToken se devuelve al cliente, pero no participa en renovación; el frontend lo interpreta como id de doctor. Logout solo elimina la clave user de localStorage.

12 Roles y permisos

Los roles persistidos son ADMIN, DOCTOR y RECEPCIONISTA. La matriz distingue comportamiento de interfaz y control del servidor. Una restricción aplicada solo en el frontend no constituye autorización backend.

Funcionalidad	ADMIN	DOCTOR	RECEPCIONISTA	Control aplicado en backend
Ver navegación de usuarios	Visible	Oculta	Oculta	La ruta /configuracion requiere sesión en el frontend, sin validación específica de rol en el servidor
Crear usuario desde UI	Visible	No visible	No visible	POST /auth/register no declara autenticación
Editar usuario desde configuración	Disponible	Acceso técnicamente posible por navegación directa	Acceso técnicamente posible por navegación directa	PATCH /usuarios/:id no declara guard
Eliminar usuario	Visible	Oculto	Oculto	DELETE /usuarios/:id no declara guard
Consultar pacientes	Filtra por creador propio	Filtra por creador propio	Doctor asignado o propia cuenta	GET /pacientes devuelve todos
Gestionar citas	Agenda propia	Agenda propia	Agenda del doctor registrada en refreshToken	El CRUD /citas no declara guard
Eliminar cita	Permitido en UI	Permitido en UI	Bloqueado en UI	DELETE /citas/:id no declara guard
Eliminar paciente	Permitido si no tiene citas	Permitido si no tiene citas	Bloqueado UI	DELETE /pacientes/:id no verifica rol ni citas
Actualizar /auth/update/:id	Cualquier id	Solo propio id	Solo propio id	JwtAuthGuard y comprobación explícita

13 API REST

La API expone 25 rutas. Salvo las dos rutas marcadas con JWT, los controladores no declaran autenticación. Los DTOs CRUD sin decoradores funcionan como formas TypeScript, no como validación runtime demostrada.

13.1 Raíz y autenticación

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta observada
GET	/	Respuesta de plantilla	No	Ninguna	Hello World!
POST	/auth/register	Registra usuario y hashea contraseña	No	RegisterDto	Usuario sin password
POST	/auth/login	Valida credenciales y emite token	No	LoginDto	access_token, user y message
GET	/auth/profile	Obtiene identidad seleccionada	JWT	Bearer token	id, email, nombre, apellido y rol
PUT	/auth/update/:id	Actualiza usuario	JWT	id y UpdateUserDto	Usuario sin password

13.2 Usuarios

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/usuarios	Crea usuario	No declarada	Body Create DTO	Registro creado por Prisma
GET	/usuarios	Lista usuarios	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/usuarios/:id	Obtiene usuario por id	No declarada	id en ruta	Registro o null
PATCH	/usuarios/:id	Actualiza usuario	No declarada	id y Update DTO	Registro actualizado
DELETE	/usuarios/:id	Elimina usuario	No declarada	id en ruta	Registro eliminado

13.3 Pacientes

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/pacientes	Crea paciente	No declarada	Body Create DTO	Registro creado por Prisma
GET	/pacientes	Lista pacientes	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/pacientes/:id	Obtiene paciente por id	No declarada	id en ruta	Registro o null
PATCH	/pacientes/:id	Actualiza paciente	No declarada	id y Update DTO	Registro actualizado
DELETE	/pacientes/:id	Elimina paciente	No declarada	id en ruta	Registro eliminado

13.4 Citas

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/citas	Crea cita	No declarada	Body Create DTO	Registro creado por Prisma

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
GET	/citas	Lista citas	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/citas/:id	Obtiene cita por id	No declarada	id en ruta	Registro o null
PATCH	/citas/:id	Actualiza cita	No declarada	id y Update DTO	Registro actualizado
DELETE	/citas/:id	Elimina cita	No declarada	id en ruta	Registro eliminado

13.5 Historia clínica

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/historia-clinica	Crea historia clínica	No declarada	Body Create DTO	Registro creado por Prisma
GET	/historia-clinica	Lista historias clínicas	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/historia-clinica/:id	Obtiene historia clínica por id	No declarada	id en ruta	Registro o null
PATCH	/historia-clinica/:id	Actualiza historia clínica	No declarada	id y Update DTO	Registro actualizado
DELETE	/historia-clinica/:id	Elimina historia clínica	No declarada	id en ruta	Registro eliminado

13.6 Cuerpos principales

DTO	Campos
RegisterDto	email, password, nombre, apellido, rol, refreshToken opcional
LoginDto	email, password
CreateUsuarioDto	nombre, apellido, email, password, rol, refreshToken, lastLogin, activo
CreatePacienteDto	nombres, apellidos, dpi, telefono, email, direccion, fechaNacimiento, estado, alergias, creadoPorId
CreateCitaDto	pacienteId, usuarioId, fecha, hora, motivo, estado, notas
CreateHistoriaClinicaDto	citaId, pacienteId, descripcion, diagnostico, tratamiento, observaciones

14 Base de datos

Prisma configura un datasource PostgreSQL mediante DATABASE_URL. La migración inicial crea tres enums y cuatro tablas. Las claves foráneas usan ON DELETE RESTRICT y ON UPDATE CASCADE. No se declaran índices adicionales fuera de las restricciones únicas de email, dpi y citaId.

Modelo	Propósito verificable	Relaciones
Usuario	Cuenta, rol, credenciales y estado	Crea pacientes y atiende citas
Paciente	Datos personales y clínicos básicos	Creador, citas e historias
Cita	Fecha, hora, motivo, estado y notas	Paciente, usuario e historia opcional
HistoriaClinica	Descripción, diagnóstico, tratamiento y observaciones	Una cita única y un paciente

14.1 Enums

Enum	Valores
Rol	ADMIN, DOCTOR, RECEPCIONISTA
EstadoPaciente	ACTIVO, INACTIVO
EstadoCita	CONFIRMADA, PENDIENTE, NUEVA, FINALIZADA, CANCELADA, DISPONIBLE

15 Diccionario de datos

15.1 Usuario

Campo	Tipo	Obligatorio	Clave o regla	Descripción
id	Int	Sí	PK, autoincrement	Identificador del usuario.
nombre	String	Sí	-	Nombre.
apellido	String	Sí	-	Apellido.
email	String	Sí	Único	Correo usado por login.
password	String	Sí	-	Hash en el flujo de registro; no hay restricción DB de formato.
rol	Rol	Sí	Default RECEPCIONISTA	Rol persistido.
refreshToken	String	No	-	Valor opcional asociado al usuario.
lastLogin	DateTime	No	-	Fecha prevista para registrar el último acceso.
activo	Boolean	Sí	Default true	Habilita o bloquea login y validación JWT.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Fecha actualizada por Prisma.

Observaciones de implementación. El frontend utiliza actualmente refreshToken como identificador del doctor asignado a una recepcionista; no existe un flujo de renovación JWT. El campo lastLogin está definido en el modelo, pero el código actual no lo actualiza.

15.2 Paciente

Campo	Tipo	Obligatorio	Clave o regla	Descripción
id	Int	Sí	PK, autoincrement	Identificador.
nombres	String	Sí	-	Nombres del paciente.
apellidos	String	Sí	-	Apellidos.
dpi	String	No	Único	Identificador documental opcional.
telefono	String	Sí	-	Teléfono.
email	String	No	-	Correo opcional.
direccion	String	No	-	Dirección opcional.

Campo	Tipo	Obligatorio	Clave o regla	Descripción
fechaNacimiento	DateTime	No	-	Fecha de nacimiento.
estado	EstadoPaciente	Sí	Default ACTIVO	Estado del paciente.
alergias	String	No	-	Texto libre sobre alergias.
creadoPorId	Int	Sí	FK Usuario	Usuario creador.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Última actualización.

15.3 Cita

Campo	Tipo	Obligatorio	Clave o regla	Descripción
id	Int	Sí	PK, autoincrement	Identificador.
pacienteId	Int	Sí	FK Paciente	Paciente asociado.
usuarioId	Int	Sí	FK Usuario	Usuario asociado a la agenda.
fecha	DateTime	Sí	-	Fecha y hora construidas por el frontend.
hora	String	Sí	-	Hora separada en HH:MM desde el frontend.
motivo	String	Sí	-	Motivo de consulta.
estado	EstadoCita	Sí	Default NUEVA	Estado operativo.
notas	String	No	-	Notas libres.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Última actualización.

15.4 HistoriaClinica

Campo	Tipo	Obligatorio	Clave o regla	Descripción
id	Int	Sí	PK, autoincrement	Identificador.
citaId	Int	Sí	FK Cita, único	Vincula como máximo una historia a cada cita.
pacienteId	Int	Sí	FK Paciente	Paciente asociado.
descripcion	String	No	-	Descripción clínica libre.
diagnostico	String	No	-	Diagnóstico libre.
tratamiento	String	No	-	Tratamiento libre.
observaciones	String	No	-	Observaciones libres.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Última actualización.

16 Modelo entidad relación

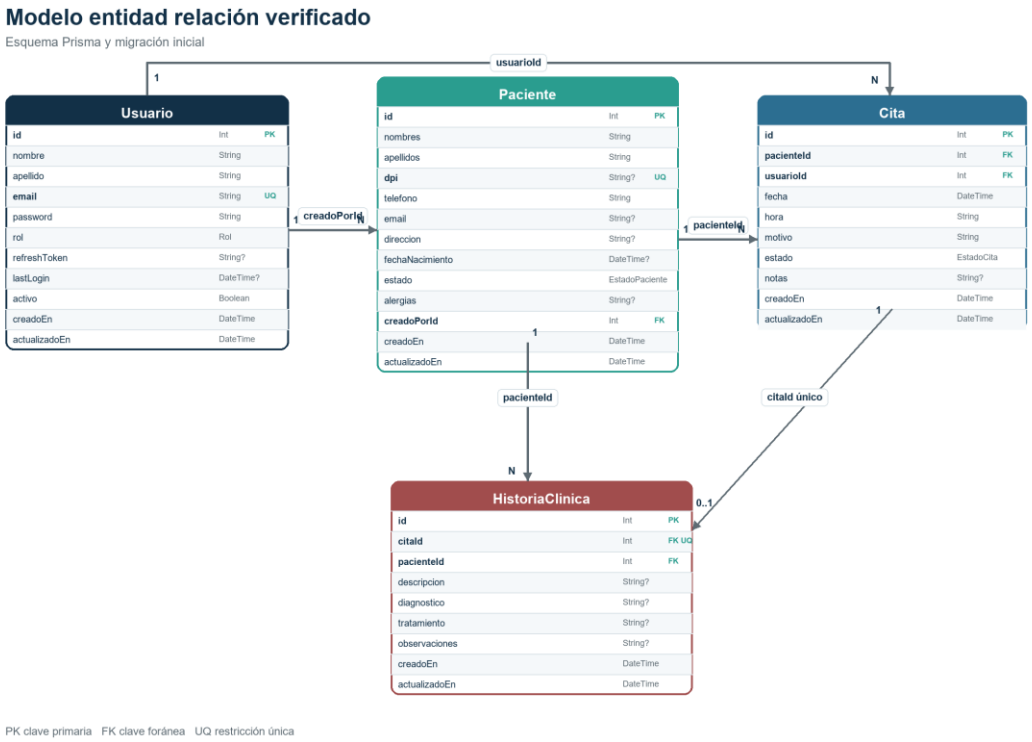


Figura 3. Modelo entidad relación derivado de schema.prisma y la migración inicial.

La unicidad de HistoriaClinica.citaId establece una relación Cita 1 a HistoriaClinica 0..1. Paciente y Usuario mantienen relaciones uno a muchos con citas; Usuario también mantiene una relación uno a muchos con pacientes creados.

17 Flujos principales del sistema

17.1 Inicio de sesión

Paso	Capa o participante	Acción
1	Actor	El usuario introduce correo y contraseña.
2	Interfaz	LoginComponent valida los campos con Angular Validators.
3	Servicio frontend	AuthService prepara la solicitud de autenticación.
4	Solicitud HTTP	POST /auth/login envía LoginDto.
5	Controller y service	AuthController delega en AuthService, que busca el usuario, verifica activo y compara la contraseña con bcrypt.
6	Prisma y base de datos	Prisma consulta el modelo Usuario en PostgreSQL.
7	Respuesta	JwtService firma el token; el frontend guarda la respuesta en localStorage y navega al dashboard.

17.2 Alta de paciente

Paso	Capa o participante	Acción
1	Actor	Un usuario con sesión abierta completa el formulario del paciente.
2	Interfaz	PacientesComponent valida los campos disponibles.
3	Servicio frontend	PacientesService transforma la fecha y añade creadoPorId desde el usuario local.
4	Solicitud HTTP	POST /pacientes envía CreatePacienteDto.
5	Controller y service	PacientesController delega la creación a PacientesService.
6	Prisma y base de datos	prisma.paciente.create inserta el registro en PostgreSQL.
7	Respuesta	La API devuelve el paciente creado y la interfaz actualiza su listado.

17.3 Programación de cita

Paso	Capa o participante	Acción
1	Actor	El usuario selecciona un paciente y completa fecha, hora y motivo.
2	Interfaz	AgendaComponent comprueba que exista un paciente seleccionado.
3	Servicio frontend	AgendaService determina usuarioId desde el usuario actual o desde refreshToken para recepcionistas y transforma la fecha.
4	Solicitud HTTP	POST /citas envía CreateCitaDto.
5	Controller y service	CitasController delega en CitasService.
6	Prisma y base de datos	Prisma persiste la cita y sus claves foráneas.
7	Respuesta	La API devuelve el registro y el frontend consulta los datos del paciente para completar su nombre.

17.4 Consulta de historia

Paso	Capa o participante	Acción
1	Actor	El usuario selecciona un paciente en HistoriaClinicaComponent.
2	Servicio frontend	HistoriaClinicaService solicita /pacientes/:id y /citas.
3	API	PacientesController y CitasController devuelven los datos consultados mediante sus servicios.
4	Prisma y base de datos	Prisma obtiene Paciente y Cita desde PostgreSQL.
5	Procesamiento cliente	El servicio filtra las citas por paciente y doctor, normaliza fecha y hora y agrupa los resultados.
6	Respuesta visual	La vista presenta un historial basado en citas y notas. Este flujo no consulta /historia-clinica.

17.5 Generación de reporte

Paso	Capa o participante	Acción
1	Actor	El usuario selecciona tipo de reporte y filtros.
2	Interfaz	ReportesComponent prepara los parámetros.
3	Servicio frontend	ReportesService opera con useClientAggregation en true.
4	Solicitudes HTTP	El servicio reutiliza PacientesService, AgendaService o HistoriaClinicaService.
5	Procesamiento cliente	Las filas se consolidan y filtran en memoria.
6	Resultado	El reporte se presenta en pantalla y puede exportarse como CSV, XLSX o PDF.

18 Seguridad

18.1 Controles implementados

Control	Implementación comprobada	Alcance
Hash de contraseña	bcrypt con costo 10 en registro y actualizaciones que pasan por AuthService o UsuariosService.update	No se aplica en UsuariosService.create
JWT	Bearer token, expiración comprobada y usuario activo en JwtStrategy	Solo rutas que usan JwtAuthGuard
Guard frontend	Comprueba existencia de access_token	No valida firma ni expiración
CORS	Lista explícita de tres orígenes y métodos	Configuración global Nest
Validadores declarativos	class-validator en LoginDto, RegisterDto y UpdateUserDto	Sin ValidationPipe global demostrado
Restricciones DB	Claves, foráneas, enums y únicos	PostgreSQL mediante migración
CSV	Prefija celdas que empiezan con = + - @	Exportación CSV de reportes

18.2 Consideraciones de seguridad

Aspecto	Implementación actual	Consideración técnica	Mejora recomendada
Autorización de endpoints	Los CRUD de usuarios, pacientes, citas e historia clínica no aplican UseGuards.	La autorización depende principalmente de controles del frontend.	Aplicar JwtAuthGuard y reglas de rol en el servidor.
Respuesta de usuarios	GET /usuarios usa findMany sin una selección que excluya campos sensibles.	La respuesta puede incluir password y refreshToken.	Definir DTOs de salida o select explícito.
Creación de usuarios	UsuariosService.create persiste el DTO directamente y lo registra en consola.	Ese flujo no aplica hash a password y puede registrar datos sensibles.	Unificar la creación con el flujo seguro de registro y retirar logs sensibles.
Configuración JWT	JwtStrategy contempla un valor alternativo y AuthModule usa una configuración distinta.	La firma puede depender de valores no alineados cuando falta JWT_SECRET.	Exigir JWT_SECRET y centralizar la configuración.
Alcance de datos	Los servicios Angular filtran creadoPorId y usuarioId.	Un cliente distinto podría omitir esos filtros.	Aplicar filtros autorizados en consultas Prisma.

Aspecto	Implementación actual	Consideración técnica	Mejora recomendada
Almacenamiento del token	La respuesta de autenticación se guarda en localStorage.	El token queda accesible al contexto JavaScript de la aplicación.	Evaluar cookies HttpOnly o reforzar controles frente a XSS.
Validación de DTO	Existen decoradores en DTOs de autenticación, sin ValidationPipe global.	Los decoradores no se aplican automáticamente a todas las solicitudes.	Registrar ValidationPipe y ampliar validación a DTOs CRUD.
Uso de refreshToken	El frontend lo interpreta como identificador de doctor.	El nombre no corresponde a su uso operativo y dificulta incorporar renovación JWT.	Crear una relación explícita doctor recepcionista.
Consistencia de roles	UpdateUserDto declara ASISTENTE en lugar de RECEPCIONISTA.	La diferencia afectaría la validación si el pipe se activa.	Alinear el enum con Prisma y el frontend.
Diagnóstico de errores	HistoriaClinicaService retorna estructuras vacías mediante catchError.	La vista conserva continuidad, pero pierde detalle sobre la causa.	Registrar el error de forma segura y mostrar un estado diferenciable.

19 Configuración del sistema

Archivo	Configuración relevante
frontend/angular.json	Build production/development, assets, Bootstrap CSS, Karma y builder angular-cli-ghpages.
frontend/tsconfig.json	TypeScript estricto, target ES2022 y strictTemplates.
backend/nest-cli.json	sourceRoot src y borrado de dist antes de compilar.
backend/tsconfig.json	NodeNext, target ES2023, decoradores y salida dist.
backend/src/main.ts	CORS y PORT con default 3000.
backend/prisma/schema.prisma	PostgreSQL, DATABASE_URL, modelos y enums.

El repositorio actual no incluye archivos .env ni .env.example. Los valores sensibles deben proporcionarse fuera del control de versiones.

20 Variables de entorno

Variable	Componente	Propósito	Obligatoria según código
DATABASE_URL	Backend Prisma	Cadena de conexión PostgreSQL	Sí para conectar y migrar
JWT_SECRET	Backend Auth	Firma y validación de JWT	La implementación contempla un valor alternativo; en producción debe exigirse explícitamente
PORT	Backend	Puerto de escucha	No; default 3000

21 Instalación y ejecución local

21.1 Prerrequisitos verificables

- Node.js y npm; el repositorio no fija versión mediante engines, .nvmrc o archivo equivalente.
- Una instancia PostgreSQL accesible mediante DATABASE_URL.

- Un navegador compatible para el frontend; las pruebas Karma están configuradas para Chrome.

21.2 Backend

```
cd backend
npm install
npx prisma generate
npx prisma migrate dev --name init
npm run start:dev
```

Los comandos Prisma aparecen comentados en schema.prisma. Antes de iniciar se deben proporcionar DATABASE_URL y JWT_SECRET sin registrar sus valores en Git.

21.3 Frontend

```
cd frontend
npm install
npm start
```

El servidor de desarrollo Angular usa normalmente http://localhost:4200, origen permitido por CORS. Sin embargo, los servicios activos apuntan a https://odonto-clinic.onrender.com; las URLs locales aparecen comentadas. El repositorio no implementa una configuración environment para alternar endpoints sin editar código.

21.4 Build

```
cd backend && npm run build
cd frontend && npm run build
```

22 Scripts disponibles

22.1 Frontend

Script	Comando	Uso
start	ng serve	Servidor de desarrollo
build	ng build	Build de producción por defecto
watch	ng build --watch --configuration development	Compilación observada
test	ng test	Karma/Jasmine

22.2 Backend

Script	Comando	Uso
build	nest build	Compilar API
start	nest start	Inicio normal
start:dev	nest start --watch	Desarrollo
start:debug	nest start --debug --watch	Depuración
start:prod	node dist/main	Ejecutar build

Script	Comando	Uso
lint	eslint ... --fix	Lint con cambios automáticos
test	jest	Unitarias
test:e2e	jest --config ./test/jest-e2e.json	E2E
test:cov	jest --coverage	Cobertura
format	prettier --write ...	Formateo

23 Despliegue

angular.json configura angular-cli-ghpages como builder de despliegue. El frontend consume una URL onrender.com para la API y main.ts autoriza un origen github.io. Estas referencias permiten identificar parcialmente la infraestructura utilizada. El repositorio no incluye workflows, Dockerfiles, render.yaml, configuración de dominio, comandos de release ni documentación suficiente para reconstruir un despliegue completo y repetible.

El README del backend conserva recomendaciones genéricas del template NestJS, incluida una opción Mau. Esa referencia pertenece a la plantilla y no permite afirmar que OdontoClinic se despliegue mediante ese servicio.

24 Manejo de errores

NestJS utiliza excepciones estándar en autenticación, como UnauthorizedException y ConflictException. En los CRUD, Prisma y el manejador predeterminado de Nest transforman los errores. La configuración actual no incorpora filtros personalizados ni un contrato uniforme de respuesta de error.

El frontend muestra mensajes específicos en login y formularios, pero varias comparaciones buscan err.message locales que no necesariamente coinciden con respuestas HTTP. catchError se usa para degradar algunas consultas a arreglos vacíos, y los componentes registran ciertos errores con console.error.

25 Validaciones

Capa	Validación observada	Limitación
Formularios Angular	required, email, minlength y selección de paciente	Solo cliente
DTO auth	IsEmail, IsString, IsNotEmpty, MinLength, IsEnum e IsOptional	Sin ValidationPipe global
DTO CRUD	Tipos TypeScript y enums locales	Sin decoradores class-validator
Servicios	Duplicado de email, usuario activo, password y propiedad de update auth	No cubre CRUD público
Prisma/PostgreSQL	Tipos, no nulos, únicos, claves foráneas y enums	No expresa todas las reglas de interfaz

26 Pruebas

El repositorio contiene 23 archivos .spec.ts y una prueba e2e. La mayoría de las pruebas verifican que un componente, controlador o servicio pueda crearse. La prueba unitaria de AppController y la prueba e2e comprueban la respuesta Hello World!.

Validación ejecutada	Resultado el 5 de octubre de 2026	Detalle
Frontend npm run build	Aprobada	Generó dist/odonto_clinik; mostró advertencias CommonJS y selectores CSS de Bootstrap.
Frontend npm test -- --watch=false	No ejecutada	Karma inició, pero no encontró Chrome en la ruta configurada.
Backend npm run build	No ejecutada	El comando nest no estaba disponible porque las dependencias backend no estaban instaladas.
Backend npm test -- --runInBand	No ejecutada	El comando jest no estaba disponible por la misma condición.

Los resultados corresponden al entorno utilizado durante la elaboración de esta documentación y no implican necesariamente un fallo del sistema. El build frontend confirma que el grafo importado de la aplicación compila en ese entorno; las pruebas no ejecutadas requieren completar las dependencias o herramientas indicadas.

27 Dependencias principales

Componente	Dependencias relevantes	Finalidad
Frontend	@angular/*, rxjs, zone.js	Framework, enrutamiento, formularios y reactividad
Frontend UI	bootstrap	Estilos y comportamiento de modales
Frontend reportes	exceljs, file-saver, jspdf, jspdf-autotable	Exportaciones
Backend	@nestjs/common, core, platform-express, config	API y configuración
Backend auth	@nestjs/jwt, passport, passport-jwt, bcrypt	JWT y contraseñas
Backend datos	@prisma/client, prisma	ORM, cliente y migraciones
Validación	class-validator, class-transformer	Decoradores DTO; falta activar pipe global

28 Consideraciones de mantenimiento

- Mantener sincronizados schema.prisma, migraciones, DTOs y modelos frontend; hoy existen enums inconsistentes.
- Centralizar la URL de API para evitar editar varios servicios al cambiar de ambiente.
- Planificar la sustitución del uso semántico de refreshToken, que actualmente almacena una relación de asignación clínica.
- Recordar que los reportes dependen de múltiples consultas del cliente y del filtrado local.
- Considerar que HistoriaClinica de base de datos y la vista basada en citas son implementaciones diferentes en el estado actual.
- Los README son plantillas de Angular y NestJS y no describen por completo la configuración real.

- Antes de actualizar dependencias de reportes, repetir el build porque Angular reporta optimizaciones CommonJS.

29 Aspectos técnicos pendientes de fortalecimiento

Aspecto	Estado actual
Autorización no centralizada	Los filtros de alcance y bloqueos por rol están en Angular; los CRUD backend no declaran guards.
Consultas no filtradas en servidor	Los controladores listan todos los registros y no consumen query params.
Historia clínica incompleta de extremo a extremo	La entidad existe, pero el frontend usa citas.
Configuración por ambiente ausente	Las URLs están codificadas y no hay environments ni .env.example.
Cobertura de pruebas	La mayoría comprueba creación; no cubre reglas, autenticación ni CRUD.
Despliegue reproducible	No hay CI/CD ni un manifiesto de despliegue para el backend.
Contrato de errores no uniforme	Mezcla excepciones Nest, Prisma y valores vacíos en frontend.

30 Recomendaciones técnicas

Prioridad	Recomendación	Resultado esperado
Alta	Aplicar JwtAuthGuard global o por controlador y RolesGuard con una matriz de permisos definida en backend.	Autorización independiente del cliente
Alta	Eliminar POST /usuarios o hacerlo equivalente al registro seguro; excluir password y refreshToken de respuestas y logs.	Reducir exposición de credenciales
Alta	Exigir JWT_SECRET sin valor alternativo y rotar cualquier valor usado fuera del gestor de secretos.	Firma coherente y no predecible
Alta	Activar ValidationPipe con whitelist, transform y forbidNonWhitelisted; añadir decoradores a DTOs CRUD.	Validación runtime consistente
Media	Crear una relación explícita doctor recepcionista en Prisma y reservar refreshToken para autenticación o eliminarlo.	Modelo semántico claro
Media	Mover filtros de usuario, fecha, búsqueda y paciente a consultas Prisma autorizadas.	Menor exposición y mejor rendimiento
Media	Integrar el módulo HistoriaClinica en el frontend o retirar el CRUD hasta definir el flujo clínico.	Consistencia funcional
Media	Centralizar API base URL en environments y agregar .env.example sin secretos.	Ejecución local y despliegue repetibles
Evolutiva	Agregar pruebas de AuthService, guards, permisos, DTOs, CRUD, filtros y escenarios e2e con base aislada.	Cobertura de reglas relevantes
Evolutiva	Añadir CI que instale con npm ci, genere Prisma, compile y ejecute pruebas.	Validación automática del repositorio
Evolutiva	Agregar logging estructurado sin datos sensibles y un filtro uniforme de excepciones.	Diagnóstico operativo seguro
Evolutiva	Evaluar cookies HttpOnly o controles XSS reforzados para el token según el modelo de despliegue.	Menor exposición del token

31 Información pendiente de documentación

- Credenciales y procedimientos de provisión de los ambientes, que no deben almacenarse en el repositorio.
- Versión operativa de Node.js y PostgreSQL aprobada para producción.
- Procedimiento exacto de despliegue de frontend y backend, dominios, responsables y rollback.
- Política institucional de roles y permisos que deba aplicar el backend.
- Definición funcional de historia clínica, diagnóstico y tratamiento en la interfaz.
- Política de respaldo, retención, cifrado y recuperación de la base de datos.
- Requisitos legales y de privacidad aplicables a datos personales y clínicos.
- Datos académicos de portada: estudiante, docente, sede y otros campos institucionales.

32 Conclusiones técnicas

OdontoClinic proporciona una base funcional para la gestión odontológica mediante una aplicación Angular conectada a una API NestJS. El sistema administra usuarios, roles, pacientes y citas, ofrece autenticación con JWT, genera reportes en distintos formatos y persiste cuatro modelos relacionados con Prisma y PostgreSQL. Su organización modular permite reconocer con claridad las responsabilidades del frontend, la API y la capa de datos.

El estado actual también presenta consideraciones que deben tenerse en cuenta durante el mantenimiento. La cobertura de autenticación del backend se concentra en dos rutas, mientras que los CRUD restantes no declaran guards; la vista de historia clínica se construye con citas en lugar de consumir la entidad HistoriaClinica; y refreshToken cumple una función operativa diferente de la que su nombre sugiere. La validación global, la cobertura de pruebas y la configuración reproducible de ambientes también pueden ampliarse.

OdontoClinic dispone de una base funcional y modular que puede continuar fortaleciéndose mediante autorización y validación en el servidor, pruebas automatizadas, configuración por ambiente y procedimientos de despliegue reproducibles. Estas mejoras permitirían conservar las funciones ya implementadas y facilitar una evolución técnica más segura y mantenible.

Anexo A Trazabilidad de evidencia

Afirmación	Archivos que la demuestran
Angular standalone con router e interceptor	frontend/src/main.ts; app/app.config.ts; app/app.routes.ts
API NestJS modular	backend/src/app.module.ts y módulos de dominio
PostgreSQL con Prisma	backend/prisma/schema.prisma; migration.sql
JWT Bearer	backend/src/auth/auth.module.ts; strategies/jwt.strategy.ts; guards/jwt-auth.guard.ts
Rutas que aplican JwtAuthGuard	backend/src/auth/auth.controller.ts; búsqueda de UseGuards en backend/src
Roles persistidos	schema.prisma enum Rol; frontend configuracion.service.ts
Reportes agregados en cliente	frontend/src/app/reportes/reportes.service.ts
Historia frontend basada en citas	frontend/src/app/historia-clinica/historia-clinica.service.ts
URL API codificada	auth.service.ts y servicios de dominio frontend

Afirmación	Archivos que la demuestran
Variables de entorno	schema.prisma; auth.module.ts; jwt.strategy.ts; main.ts
Pruebas existentes	archivos *.spec.ts; backend/test/app.e2e-spec.ts; package.json
Deploy gh-pages configurado	frontend/angular.json

Anexo B Resumen cuantitativo del proyecto

Elemento	Cantidad o estado
Archivos del repositorio excluyendo dependencias y artefactos	134
Archivos TypeScript backend	50
Archivos TypeScript frontend	36
Componentes frontend no test	10
Servicios frontend no test	7
Controladores backend	6
Módulos backend	7
Servicios backend	7
Endpoints HTTP	25
Modelos Prisma	4
Archivos spec	23
Build frontend	Aprobado con advertencias
Build y pruebas backend	No ejecutables sin dependencias instaladas